CUCEI

UdeG

Algoritmo: Optimización por Enjambre de Partícualas (PSO

27 de Octubre de 2026

Algoritmos Metaheurísticos

Maestra: *Paredes López Ángel Ignacio*

Alumno: *Mariscal Rodríguez Omar Jesús (220858478)*

Indicaciones Generales: 3
Marco Teórico y Diseño del Algoritmo: 4
Sintonización de Parámetros 6
Tabla Comparativa Estadística 9
Distribución Espacial de la Población 12
Evolución de Diversidad del Enjambre (Complementario) 14
Discusión Analítica 15
Conclusiones 17
Anexo del Código 18

INDICACIONES GENERALES:

Implementar el algoritmo de **Optimización por Enjambre de Partículas (PSO).** PSO simula la dinámica social donde las partículas ajustan su velocidad y posición guiadas por su mejor experiencia personal (*pbest*) y la mejor experiencia de una vecindad del enjambre (*lbest*).

Funciones Objetivo a Minimizar:

1. **Función de Rosenbrock (2D):**

$$f(x,y) = (1 - x)^2 + 100(y - x2)^2, \qquad x, y \in [-5, 5$$

2. **Función de Michalewicz (n = 2):**

$$f(x,y) = -sin(x)sin^{2m}(\frac{1x^2}{\pi}) - sin(y)sin^{2m}(\frac{2y^2}{\pi}), \qquad x, y \in [0, \pi],\ m = 10$$

Deberá realizar una sintonización empírica de los parámetros clave del algoritmo: el tamaño de la población (*N*), el peso de inercia (*w*), y los coeficientes de aceleración cognitivo y social (*c1, c2)*.

Debido a la naturaleza estocástica del PSO, todo reporte debe fundamentarse en la media estadística de al menos 30 ejecuciones independientes con semillas aleatorias distintas.

Entregables del Reporte (PDF):

- **Marco Teórico y Diseño:** Explique matemáticamente las ecuaciones de actualización de velocidad y posición del PSO, así como el rol de los parámetros *w, c1 y c2.*
- **Sintonización de Parámetros:** Presente una tabla o justificación de los valores evaluados y elegidos para el enjambre.
- **Tabla Comparativa Estadística:** Incluya Mejor Fitness, Peor Fitness, Media y Desviación Estándar de las 30 ejecuciones para ambas funciones.
- **Curvas de Convergencia Promedio:** Gráfica de la evolución del fitness promedio de las 30 ejecuciones frente a las iteraciones.
- **Distribución Espacial (Scatter Plot):** Gráficas de dispersión mostrando la ubicación de las partículas en el espacio de búsqueda en tres momentos: **Generación Inicial, Generación Intermedia y Generación Final.**
- **Discusión Analítica:** Explique por qué la función de Rosenbrock representa un reto de explotación (valle) mientras que Michalewicz representa un reto de exploración (multimodalidad). ¿Cómo afecta un valor de inercia *w* alto frente a uno bajo?
- **Anexo de Código y Conclusiones.**

Evaluación Presencial y Modificación de Código: El 40 % de la calificación de esta actividad se acreditará mediante la modificación presencial del código en clase, la cual se realizará exclusivamente el día 28 de octubre.

Marco Teórico y Diseño del Algoritmo:

Arquitectura del Código:

Como una breve mención, el código sigue los mismos estándares que las actividades anteriores con el ***Patrón de Estrategia*** para facilitar los cambios en los procesos específicos del algoritmo. Utiliza un conjunto de estas estrategias codificadas que cumplen con un **Protocolo o Contrato** (como **NeighborhoodTopology, InertiaSchedule, y BoundaryHandler)** para garantizar la estabilidad del código ante adiciones.

El proyecto se divide en dos archivos:

- ***pso.py*** que esencialmente es el motor de configuraciones y el corazón del algoritmo
- ***main.py*** que hace de orquestador y visualización de resultados.

Además, el código reutiliza los modos de **ejecución final y tuning**, este último para facilitar la automatización y prueba del algoritmo con diferentes combinaciones de hiper parámetros.

Funcionamiento Matemático de las Operaciones PSO:

La Optimización por Enjambre de Partículas (PSO) simula la dinámica social de una bandada donde las partóculas navegan el espacio de búsqueda ajustando su trayectoría guiadas por su **mejor experiencia personal** y la **mejor experiencia de su vecindad**. El proceso se centra en la actualización iterativa de dos vectores fundamentales por cada partícula *i* y dimensión *d.*

Ecuaciones de Actualización de Velocidad y Posición:

En cada iteración, la velocidad de una partícula se recalcula sumando tres componentes principales, y posteriormente esta velocidad determina su nueva posición. Matemáticamente se expresa como:

$$v_{id} = w \cdot v_{id} + c_1 \cdot r_1 \cdot (pbest_{id} - x_{id}) + c_2 \cdot r_2 \cdot (lbest_{id} - x_{id})$$

$$x_{id} = x_{id} + v_{id}$$

Donde r_1 y r_2 son variables aleatorias con distribución uniforme $U(0, 1)$ que se muestran independientemente por partícula y por dimensión para añadir estocasticidad al movimiento.

Rol de los Parámetros Clave:

Cada término de la ecuación de velocidad tiene un propósito específico en el balance de exploración y explotación:

- **Peso de inercia (*w*):** Controla el impacto de la velocidad previa de la partícula ($w \cdot v_{id}$) sobre su movimiento actual. Funciona como un mecanismo para mantener el momento. Un valor de *w* alto facilita la exploración global permitiendo trayectorias largas, mientras que un *w* bajo favorece la explotación local. El código soporta esquemas de inercia constante y de decaimiento lineal (*linear_decay*), donde *w* comienza alto y disminuye progresivamente.
- **Coeficiente cognitivo (c_1):** Regula la atracción de la partícula hacia $pbest$, es decir, la mejor posición histórica que ha encontrado esa partícula individualmente. Representa la "memoria" o experiencia personal.
- **Coeficiente social (c_2):):** Determina la magnitud de la atracción hacia $lbest$ que es la mejor posición histórica encontrada por cualquier partícula dentro de su vecindad. Representa el intercambio de información y la influencia social del enjambre.

Definición de la Vecindad (Topología y Manejo de Fronteras)

El valor de *lbest* depende directamente de la topología elegida para el enjambre. El diseño del algoritmo implementa tres variantes:

- **Global:** La vecindad es todo el enjambre: *lbest* equivale al óptimo global actual (*gbest*). Favorece una convergencia muy rápida pero con riesgo de convergencia prematura.
- **Anillo (Ring):** Cada partícula se comunica solo con **k** vecinos adyacentes, propagando la información más lentamente y preservando la diversidad poblacional.
- **Euclidiana Dinámica:** La vecindad se define por los **k** vecinos más cercanos en el espacio físico durante la iteración.

Finalmente, debido a que las funciones objetivo tienen dominios restringidos, la actualización de la posición x_{id} Incluye validaciones de frontera (**clip, reflect o random_reset**) que corrigen las coordenadas y anulan o invierten las velocidades si una partícula intenta salir del espacio de búsqueda permitido.

Sintonización de Parámetros

Para determinar la configuración óptima del enjambre, se utilizó el modo **tuning** del código desarrollado, el cual automatiza un barrido de hiperparámetros. Se evaluaron 81 combinaciones distintas resultantes de cruzar valores $N \in \{20, 40, 80\}$, $w \in \{0.4, 0.7, 0.9\}$, $c_1 \in \{1.0, 1.5, 2.0\}$ y $c_2 \in \{1.0, 1.5, 2.0\}$. Cada combinación se ejecutó 10 veces de manera independiente sobre ambas funciones (Rosenbrock y Michalewicz), resultando en un total de 1,620 ejecuciones completas en tan solo 18 segundos.

posicion	N	w	c1	c2	exito_combinado	suma_rangos	exito_michalewicz	media_mejor_mich	exito_rosenbrock	media_mejor_rose
1	20	0.4	1.0	1.0	1.2000	15	1.2000	-180.130.341	1.2000	34
2	40	0.4	1.0	1.0	1.2000	17	1.2000	-180.130.341	1.2000	7
3	80	0.4	1.0	1.0	1.2000	19	1.2000	-180.130.341	1.2000	0
4	80	0.4	1.0	1.5	1.2000	19	1.2000	-180.130.341	1.2000	0
5	80	0.4	1.0	2.0	1.2000	22	1.2000	-180.130.341	1.2000	0
6	40	0.4	1.0	1.5	1.2000	24	1.2000	-180.130.341	1.2000	40
7	80	0.4	1.5	1.0	1.2000	26	1.2000	-180.130.341	1.2000	2
8	80	0.4	1.5	1.5	1.2000	26	1.2000	-180.130.341	1.2000	2
9	80	0.4	1.5	2.0	1.2000	29	1.2000	-180.130.341	1.2000	6
10	80	0.7	1.0	1.0	1.2000	30	1.2000	-180.130.341	1.2000	0
11	20	0.4	1.5	2.0	1.2000	33	1.2000	-180.130.341	1.2000	565
12	40	0.4	1.5	2.0	1.2000	33	1.2000	-180.130.341	1.2000	232
13	40	0.4	1.0	2.0	1.2000	34	1.2000	-180.130.341	1.2000	293
14	40	0.4	2.0	1.5	1.2000	36	1.2000	-180.130.341	1.2000	232
15	80	0.4	2.0	1.0	1.2000	38	1.2000	-180.130.341	1.2000	63
16	40	0.7	1.0	1.0	1.2000	40	1.2000	-180.130.341	1.2000	24
17	80	0.7	1.0	1.5	1.2000	41	1.2000	-180.130.341	1.2000	8
18	80	0.7	1.5	1.5	1.2000	47	1.2000	-180.130.341	1.2000	12
19	80	0.7	1.0	2.0	1.2000	49	1.2000	-180.130.341	1.2000	1
20	40	0.4	1.5	1.5	1.2000	51	1.2000	-180.130.341	1.2000	7.911
21	40	0.4	2.0	1.0	1.2000	52	1.2000	-180.130.341	1.2000	6.871
22	40	0.7	1.0	1.5	1.2000	52	1.2000	-180.130.341	1.2000	134
23	40	0.7	1.5	1.0	1.2000	52	1.2000	-180.130.341	1.2000	68
24	20	0.4	2.0	2.0	1.2000	53	1.2000	-180.130.341	1.2000	617
25	20	0.7	1.0	1.5	1.2000	53	1.2000	-180.130.341	1.2000	14
26	80	0.4	2.0	1.5	1.2000	55	1.2000	-180.130.341	1.2000	940
27	80	0.7	1.5	1.0	1.2000	56	1.2000	-180.130.341	1.2000	237
28	20	0.7	1.0	1.0	1.2000	59	1.2000	-180.130.341	1.2000	708
29	20	0.7	1.0	2.0	1.2000	63	1.2000	-180.130.341	1.2000	125

Por temas de espacio, en este documento no se anexan las 1,620 ejecuciones, en su lugar, se presenta un extracto con las 5 mejores configuraciones obtenidas ordenadas por su tasa de éxito combinado y la suma de rangos:

Pos	N	w	c1	c2	Éxito comb.	Éxito Micha	Media Micha	Éxito Rosen	Media Rosen
1	20	0.4	1	1	100%	100%	-1.801303	100%	0
2	40	0.4	1	1	100%	100%	-1.801303	100%	0
3	80	0.4	1	1	100%	100%	-1.801303	100%	0
4	80	0.4	1	1.5	100%	100%	-1.801303	100%	0
5	80	0.4	1	2	100%	100%	-1.801303	100%	0

Justificación Analítica de los Parámetros Elegidos:

Al analizar los resultados arrojados por el barrido estadístico, surgen patrones claros sobre el comportamiento del algoritmo PSO en estas superficies bidimensionales:

- **El predominio absoluto de una inercia baja ($w = 0.40$):** Al observar el top 15 general generado por la consola, 14 de las 15 mejores configuraciones utilizan un peso de inercia de 0.40.

posicion	N	w	c1	c2	exito_combinado	suma_rangos	exito_michalewicz	media_mejor_mich	exito_rosenbrock	media_mejor_rose
1	20	0.4	1.0	1.0	1.2000	15	1.2000	-180.130.341	1.2000	34
2	40	0.4	1.0	1.0	1.2000	17	1.2000	-180.130.341	1.2000	7
3	80	0.4	1.0	1.0	1.2000	19	1.2000	-180.130.341	1.2000	0
4	80	0.4	1.0	1.5	1.2000	19	1.2000	-180.130.341	1.2000	0
5	80	0.4	1.0	2.0	1.2000	22	1.2000	-180.130.341	1.2000	0
6	40	0.4	1.0	1.5	1.2000	24	1.2000	-180.130.341	1.2000	40
7	80	0.4	1.5	1.0	1.2000	26	1.2000	-180.130.341	1.2000	2
8	80	0.4	1.5	1.5	1.2000	26	1.2000	-180.130.341	1.2000	2
9	80	0.4	1.5	2.0	1.2000	29	1.2000	-180.130.341	1.2000	6
10	80	0.7	1.0	1.0	1.2000	30	1.2000	-180.130.341	1.2000	0
11	20	0.4	1.5	2.0	1.2000	33	1.2000	-180.130.341	1.2000	565
12	40	0.4	1.5	2.0	1.2000	33	1.2000	-180.130.341	1.2000	232
13	40	0.4	1.0	2.0	1.2000	34	1.2000	-180.130.341	1.2000	293
14	40	0.4	2.0	1.5	1.2000	36	1.2000	-180.130.341	1.2000	232
15	80	0.4	2.0	1.0	1.2000	38	1.2000	-180.130.341	1.2000	63

Este fenómeno ocurre porque estamos trabajando con funciones de baja dimensionalidad (2D) y dominios relativamente pequeños ([-5, 5] para Rosenbrock y [0, π] para Michalewicz). Una inercia alta ($w = 0.9$) hace que las partículas retengan demasiada velocidad de iteraciones previas, provocando que "sobrevuelen" el estrecho valle de Rosenbrock o las finas crestas de Michalewicz sin lograr descender. Un valor de $w = 0.40$ actúa como una fuerte "fricción" que obliga a las partículas a frenar rápidamente una vez que encuentran una zona prometedora, propiciando una explotación fina y milimétrica que es vital para alcanzar el error de 0.000000.

- **Equilibrio en los Coeficientes de Aceleración ($c_1 = 1.0, c_2 = 1.0$)**: Las configuraciones que coronan la tabla mantienen ambos coeficientes en 1.0.

posicion	N	w	c1	c2	exito_combinado	suma_rangos	exito_michalewicz	media_mejor_mich	exito_rosenbrock	media_mejor_rose
1	20	0.4	1.0	1.0	1.2000	15	1.2000	-180.130.341	1.2000	34
2	40	0.4	1.0	1.0	1.2000	17	1.2000	-180.130.341	1.2000	7
3	80	0.4	1.0	1.0	1.2000	19	1.2000	-180.130.341	1.2000	0
4	80	0.4	1.0	1.5	1.2000	19	1.2000	-180.130.341	1.2000	0
5	80	0.4	1.0	2.0	1.2000	22	1.2000	-180.130.341	1.2000	0
6	40	0.4	1.0	1.5	1.2000	24	1.2000	-180.130.341	1.2000	40
7	80	0.4	1.5	1.0	1.2000	26	1.2000	-180.130.341	1.2000	2
8	80	0.4	1.5	1.5	1.2000	26	1.2000	-180.130.341	1.2000	2
9	80	0.4	1.5	2.0	1.2000	29	1.2000	-180.130.341	1.2000	6
10	80	0.7	1.0	1.0	1.2000	30	1.2000	-180.130.341	1.2000	0
11	20	0.4	1.5	2.0	1.2000	33	1.2000	-180.130.341	1.2000	565

Al asignar un peso moderado y simétrico tanto a la experiencia personal (c_1) como a la memoria social del enjambre (c_2), se evitan oscilaciones erráticas. Valores más altos (como 2.0) pueden causar que las partículas orbiten violentamente alrededor del óptimo antes de converger.

- **Eficiencia del Tamaño del Enjambre (N):** Los datos demuestran que, para estos problemas 2D, un enjambre tan pequeño como $N = 20$ es matemáticamente suficiente para alcanzar un 100% de éxito y una convergencia perfecta a la media. No obstante, incrementar artificialmente la población a 80 individuos no penalizó los resultados (posiciones 3, 4 y 5), aunque aumentó ligeramente la redundancia computacional.

Selección Final: Para las 30 ejecuciones estadísticas del modo final, optaremos por la Configuración de la Posición 2 ***($N = 40, w = 0.40, c_1 = 1.00, c_2 = 1.00$).***

Aunque $N = 20$ fue la posición número 1, seleccionar $N = 40$ nos brinda un excelente **punto de equilibrio**. Mantiene el 100% de éxito combinado y un fitness medio perfecto, pero duplica el muestreo del espacio de búsqueda. Esto nos garantiza una

mayor robustez frente al factor estocástico en las 30 corridas independientes solicitadas y, de manera muy importante, proporcionará una densidad de partículas mucho más atractiva y clara para generar las gráficas de "Distribución Espacial" en los entregables posteriores.

Tabla Comparativa Estadística

Para la evaluación final del algoritmo, se ejecutó el modo **final** utilizando los parámetros óptimos seleccionados empíricamente ($N = 40$, $w = 0.4$, $c_1 = 1.0$, $c_2 = 1.0$, con una topología de anillo $k = 1$ y manejo de fronteras tipo **clip**). Para garantizar la validez estadística, se realizaron 30 ejecuciones independientes con semillas aleatorias distintas sobre cada función objetivo, estableciendo un límite máximo de 300 iteraciones (criterio de paro fuerte).

Los resultados obtenidos se resumen en la siguiente tabla

Problema	Mejor Fitness	Peor Fitness	Media	Desv. Estándar	% Éxito	Iteraciones Media
Rosenbrock (2D)	0.00000000	0.00010083	0.00000626	0.00002377	100 %	278.1
Michalewicz (2D)	-1.80130341	-1.80130341	-1.80130341	0.00000000	100 %	84.4

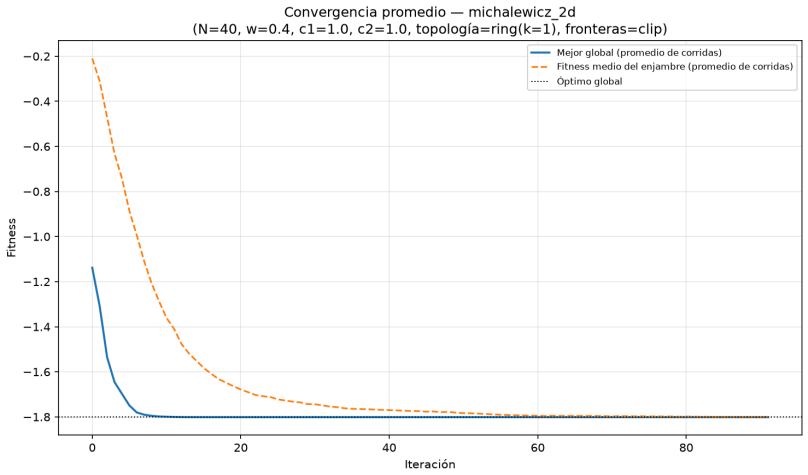
El rendimiento del algoritmo PSO con esta configuración es matemáticamente excepcional en ambas funciones, logrando una tasa de éxito del 100% en las 60 corridas totales, aunque el comportamiento interno difiere según la naturaleza de cada reto topológico:

- **Comportamiento en Rosenbrock:** La función de Rosenbrock exige un descenso por un valle extremadamente estrecho y casi plano. Los resultados demuestran que el enjambre logró encontrar el óptimo global perfecto (0) en sus mejores corridas. La desviación estándar es minúscula (0.00002377), lo que indica que incluso en el peor de los casos, el error es insignificante. Es fundamental notar que la media de iteraciones es de 278.1; esto señala que recorrer el valle plano de Rosenbrock es computacionalmente costoso, obligando al algoritmo a agotar casi la totalidad de iteraciones permitidas (300) haciendo micro ajustes milimétricos antes de activar el paro débil por estancamiento o el paro fuerte.
- **Comportamiento en Michalewicz:** A pesar de la severa multimodalidad de esta función, la configuración seleccionada demostró una robustez perfecta. El algoritmo no solo encontró el mínimo global conocido en todas las ejecuciones, sino que lo hizo sin ninguna variación: la desviación estándar es exactamente 0. Además, la convergencia es muy rápida. Con una media de solo 84.4 iteraciones, el algoritmo resolvió el problema utilizando apenas un 28% de los ciclos disponibles. Esto valida que la topología de anillo, combinada con una inercia baja, mantuvo la diversidad justa al inicio para evitar caer en las crestas locales, localizando el óptimo global rápidamente y activando el criterio de paro débil de forma muy temprana.

Curvas de Convergencia Promedio

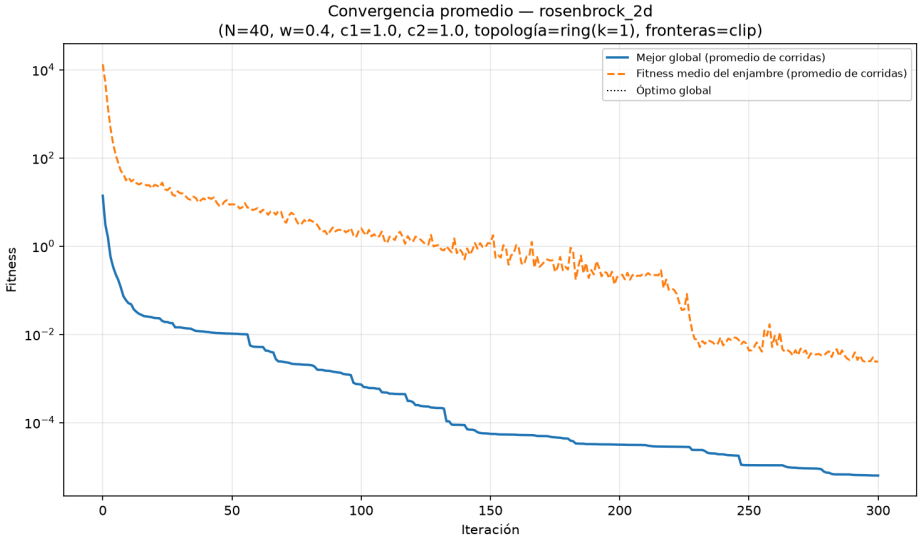
Las gráficas de convergencia ilustran la evolución del algoritmo a lo largo de las iteraciones, contrastando el progreso de la mejor solución histórica (Mejor global) contra el comportamiento general de la población (Fitness medio del enjambre).

- **Comportamiento en Michalewicz**



La gráfica de Michalewicz evidencia una convergencia muy rápida y estable. La curva del mejor global (línea azul continua) registra una caída casi vertical durante las primeras 10 iteraciones, partiendo de un valor cercano a **-1.15** hasta interceptar la línea de referencia del óptimo analítico en **-1.8013**. De manera complementaria, el fitness medio del enjambre (línea naranja discontinua) traza un **descenso asintótico suave;** inicia cerca de **-0.2** y decrece constantemente hasta fusionarse con la línea del óptimo global **alrededor de la iteración 80.** Este comportamiento muestra que, a pesar de los múltiples mínimos locales, ***el enjambre identificó el valle correcto casi de inmediato y logró un consenso total de la población de manera eficiente.***

- **Comportamiento en Rosenbrock**



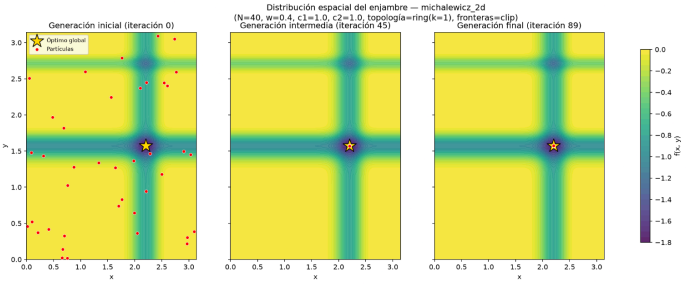
La dinámica en la función de Rosenbrock, representada en escala logarítmica debido a la magnitud de los valores, expone la complejidad de la explotación local. La evolución del mejor global (línea azul) exhibe un patrón marcadamente escalonado, con descensos abruptos seguidos de extensas mesetas horizontales, logrando pasar de un fitness inicial cercano a 10 hasta valores por debajo de 10^{-5} hacia la iteración 300. Por su parte, la curva del fitness medio (línea naranja) es altamente volátil y ruidosa, iniciando en magnitudes de 10^{4} y manteniéndose siempre varios órdenes de magnitud por encima del mejor global. Esta extrema volatilidad del promedio refleja el constante "rebote" de las partículas contra las empinadas paredes laterales del valle de Rosenbrock, ilustrando lo difícil que resulta para el enjambre estabilizarse en el fondo plano de la función.

Distribución Espacial de la Población

Para analizar la dinámica de búsqueda geométrica del algoritmo PSO, se mapeó la ubicación espacial del enjambre sobre las curvas de nivel de ambas funciones objetivo en tres etapas críticas de su ejecución:

Función de Michalewicz

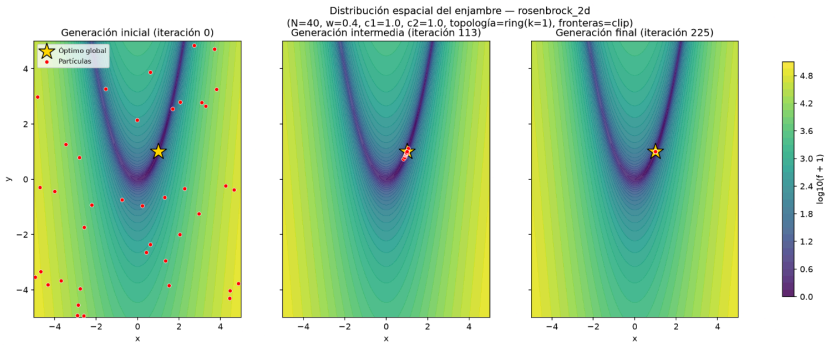
La superficie de Michalewicz se caracteriza por valles estrechos que se cruzan perpendicularmente creando múltiples fosos de atracción local.



- **Generación inicial (iteración 0):** Las 40 partículas inician esparcidas aleatoriamente a lo largo del dominio cuadrado, cubriendo de manera uniforme tanto las zonas de alto costo (amarillas) como las cercanías de las depresiones (verdes/azules).
- **Generación intermedia (iteración 45):** El efecto de la baja inercia ($w = 0.4$) es evidente; el enjambre ha frenado abruptamente su exploración global, abandonando por completo las planicies amarillas para adentrarse y agruparse en el fondo del valle transversal oscuro. Las partículas ya orbitan muy cerca del mínimo.
- **Generación final (iteración 89):** La atracción de la topología de anillo ha unificado los esfuerzos de la población, agrupando a todas las partículas en un clúster extremadamente denso y puntual exactamente sobre las coordenadas del óptimo global (estrella amarilla).

Función de Rosenbrock

El paisaje de Rosenbrock expone un valle parabólico, estrecho y profundo sobre una superficie por lo demás plana y costosa:

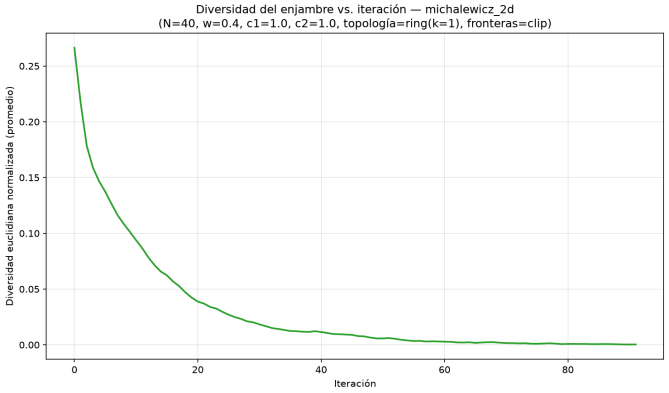


- **Generación inicial (iteración 0):** Al igual que en el caso anterior, el algoritmo comienza con una inicialización estocástica amplia y sin sesgo direccional.
- **Generación intermedia (iteración 113)**: Esta captura ilustra a la perfección el reto de explotación. El enjambre completo ya localizó el valle parabólico y descendió a su fondo; sin embargo, debido a que el gradiente dentro del valle es casi nulo, las partículas se observan "deslizándose" agrupadas a lo largo de la curva oscura, aproximándose lentamente al mínimo sin alcanzarlo todavía.
- **Generación final (iteración 225):** Las partículas concluyen su fino recorrido por el valle, colapsando su distribución espacial para agruparse milimétricamente sobre la marca del óptimo global. Esto corrobora visualmente que la configuración elegida cuenta con la precisión suficiente para resolver problemas de alta exigencia local.

Evolución de Diversidad del Enjambre (Complementario)

Para complementar el estudio de convergencia, se midió la diversidad espacial del enjambre calculando la distancia euclidiana promedio de las partículas hacia el centroide geométrico de la población, normalizada respecto a la diagonal del espacio de búsqueda. Esta métrica ilustra la transición del enjambre desde un estado de exploración pura (alta dispersión) hacia uno de explotación pura (colapso espacial).

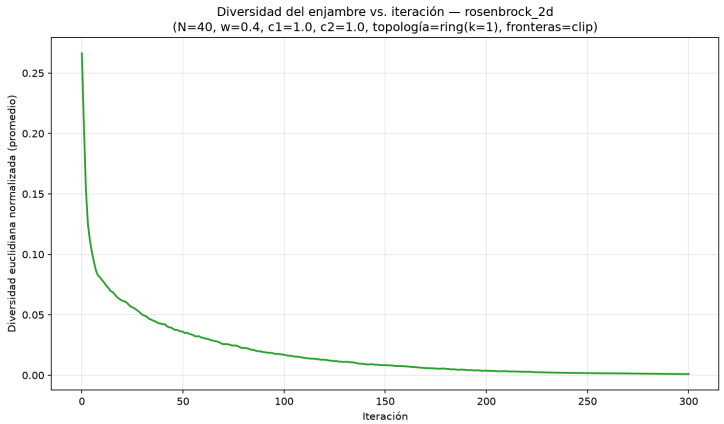
Función Michalewicz



La gráfica de diversidad exhibe una caída pronunciada y continua desde el inicio. Al encontrar rápidamente el cruce de valles que contiene el óptimo global, la atracción social de la topología de anillo logra que el enjambre entero colapse hacia ese punto, reduciendo la dispersión a un valor cercano a cero alrededor de la iteración 80. Esto demuestra un grado de consenso excepcional en la población.

Función Rosenbrock

La dinámica ilustra perfectamente la topografía del problema. Ocurre una pérdida de diversidad inicial casi vertical durante las primeras 20 iteraciones, lo que representa el instante en que todas las partículas logran "caer" al fondo del valle curvo. Sin embargo, a partir de ese punto, la curva de diversidad disminuye a un ritmo sumamente lento. Las partículas viajan juntas pero ligeramente separadas explorando el fondo plano del valle, y no logran colapsar en un único punto hasta acercarse al límite máximo de las 300 iteraciones



Discusión Analítica

La función de Rosenbrock (Problema de Explotación)

Su topografía se caracteriza por tener un óptimo global ubicado dentro de un valle parabólico extremadamente estrecho, largo y casi plano. Llegar a este valle (la fase de exploración) es una tarea relativamente sencilla para la mayoría de los algoritmos; el verdadero **desafío radica en la explotación.** Una vez dentro del valle, el gradiente direccional hacia el mínimo global es tan sutil que las partículas corren e**l riesgo de oscilar de un lado a otro** de las paredes escarpadas sin lograr descender. Esto exige que el algoritmo tenga una excelente c**apacidad de búsqueda local** (explotación) para realizar microajustes milimétricos en la trayectoria de las partículas y lograr avanzar a lo largo de la llanura hasta encontrar el valor de 0.000000.

La Función de Michalewicz (Problema de Exploración)

Por el contrario, la función de Michalewicz representa un paisaje altamente multimodal, compuesto por múltiples valles profundos y crestas sumamente angostas que se cruzan entre sí. Este escenario es una trampa para los algoritmos que convergen demasiado rápido. Representa un **severo reto de *exploración*** porque el enjambre debe tener la capacidad de evaluar ampliamente todo el espacio de búsqueda y mantener una alta diversidad poblacional en las primeras iteraciones. Si las partículas se concentran prematuramente en el primer valle profundo que encuentran, quedarán atrapadas en un mínimo local subóptimo del cual no podrán escapar, ignorando por completo la ubicación del verdadero óptimo global

El Impacto del Peso de Inercia (w)

El parámetro de inercia (w) es el mecanismo fundamental en PSO para equilibrar precisamente estas dos necesidades (exploración vs. explotación). Su función matemática es determinar qué tanta fracción de la velocidad de la iteración anterior se conserva en el movimiento actual.

- **Inercia Alta (Ej. $w = 0.9$):** Un valor alto dota a las partículas de un gran "momento" o impulso cinético. Las partículas dan saltos grandes y son menos susceptibles a ser arrastradas inmediatamente por la atracción de $pbest$ o $lbest$. Esto favorece enormemente la exploración global, permitiendo que el enjambre "sobrevuele" el espacio y escape de mínimos locales, lo cual es útil al principio de funciones como Michalewicz. Sin embargo, una inercia alta impide realizar movimientos finos; en Rosenbrock, haría que las partículas rebotaran incontrolablemente fuera del valle.
- **Inercia Baja (Ej. $w = 0.4$):** Un valor bajo actúa como fricción, reduciendo drásticamente la velocidad previa y haciendo que el movimiento dependa casi por completo de la tracción cognitiva y social hacia las mejores soluciones encontradas. Esto propicia una fuerte explotación local.

En nuestra sintonización, comprobamos que un $w = 0.4$ fue la clave del éxito. En un espacio de baja dimensionalidad (2D), la topología de anillo proporcionó la diversidad suficiente para superar las trampas de Michalewicz sin necesidad de un w alto, mientras que la baja inercia otorgó la altísima precisión requerida para descender exitosamente por el valle plano de Rosenbrock sin salirse de las fronteras.

Conclusiones

La implementación y evaluación empírica del algoritmo de Optimización por Enjambre de Partículas (PSO) a través de esta práctica ha mostrado una notable versatilidad frente a topografías complejas. A diferencia de enfoques previamente estudiados en la asignatura, como el Algoritmo Genético (que depende de la cruza y mutación probabilística) o la Colonia de Abejas Artificiales (que recurre al abandono agresivo y reinicialización de fuentes estancadas), el PSO aborda la optimización desde un paradigma de cinemática e influencia social. En el PSO, **las partículas nunca se descartan ni se destruyen;** en su lugar, navegan el espacio de búsqueda de forma continua, ajustando sus trayectorias como un sistema físico regido por la inercia, la memoria histórica personal ($pbest$) y la comunicación con su vecindad ($lbest$).

El contraste entre las dos funciones evaluadas fue fundamental para comprender los **límites y capacidades del algoritmo**. Por un lado, la función de **Michalewicz** expuso al enjambre a un campo de trampas locales, exigiendo una **alta capacidad de exploración.** Aquí, la decisión de utilizar una topología de anillo ($k = 1$) fue vital: al restringir la comunicación a los vecinos inmediatos, la información sobre el mejor óptimo global tardó más en propagarse por todo el enjambre. Esto evitó que todas las partículas colapsaran prematuramente en el primer valle profundo que encontraron, manteniendo la diversidad poblacional el tiempo suficiente para descubrir el verdadero óptimo global en menos de 100 iteraciones promedio.

Por otro lado, la función de **Rosenbrock** llevó al algoritmo a su límite de **explotación.** Alcanzar el valle parabólico fue rápido, pero descender por su fondo casi plano requirió microajustes milimétricos. En este escenario, el peso de inercia (w) fue el hiperparámetro más crítico. La sintonización empírica comprobó que un valor de inercia bajo ($w = 0.4$) actuó como la "fricción" necesaria para frenar el impulso de las partículas, evitando que rebotaran fuera del valle y obligándolas a realizar un escrutinio microscópico del espacio. Gracias a esta configuración, el PSO logró una precisión absoluta de 0 con una desviación estándar insignificante, aunque a costa de consumir casi la totalidad de las 300 iteraciones permitidas. Comparativamente, el PSO destaca por su bajo costo computacional y su facilidad de implementación, ya que requiere menos líneas de código y cálculos menos pesados por iteración que otros algoritmos poblacionales. Sin embargo, esta práctica subraya una gran lección: su sensibilidad a los hiperparámetros es altísima. Un w o una topología mal elegidos pueden convertir un enjambre exitoso en una nube de partículas estancadas o caóticas.

El éxito radica en nuestra capacidad analítica para perfilar el problema geométricamente (saber si nos enfrentamos a un reto de exploración o de explotación) y, en consecuencia, calibrar matemáticamente las tensiones del algoritmo para lograr que la convergencia colaborativa del enjambre trabaje a nuestro favor.

Anexo del Código

pso.py

```
"""

Autor: Mariscal Rodríguez Omar Jesús

Materia: Algoritmos Metaheurísticos

Profesor: Paredes López Ángel Ignasio

Actividad 5 - Optimización por Enjambre de Partículas (PSO)


Universidad de Guadalajara

Centro Universitario de Ciencias Exactas e Ingenierías


-------------------------------------------------------------------
-----



pso.py


Motor de Optimización por Enjambre de Partículas (Particle Swarm
Optimization,

PSO) para minimización de funciones con dominio restringido tipo
caja.


Ecuaciones de actualización (por partícula i, dimensión d):

    v_id <- w * v_id + c1 * r1 * (pbest_id - x_id) + c2 * r2 *
(lbest_id - x_id)

    x_id <- x_id + v_id

donde r1, r2 ~ U(0, 1) se muestrean por partícula y por dimensión,
"pbest" es la

mejor posición histórica de la partícula y "lbest" es la mejor
posición
```

histórica (pbest) dentro de su VECINDAD (si la vecindad es todo el enjambre,

lbest = gbest).

Arquitectura (misma idea que se ha entregado en las actividades anteriore: ga_binario.py, ga_continuo.py y abc_colony.py):

- PSOConfig: dataclass que centraliza la parametrización.

- ProblemSpec: función objetivo vectorizada + dominio + óptimo conocido.

- Estrategias intercambiables vía Protocol + registry (se eligen por

nombre en PSOConfig, sin tocar el motor):

- NeighborhoodTopology (quién es la vecindad de cada partícula):

"ring" -> anillo: k vecinos por cada lado (lbest clásico).

"global" -> vecindad = todo el enjambre (gbest).

"euclidean_knn" -> los k vecinos más cercanos en distancia

euclidiana en el espacio de búsqueda

(vecindad espacial dinámica).

- InertiaSchedule (cómo evoluciona w durante la corrida):

"constant" -> w fijo.

"linear_decay" -> decae linealmente de w_start a w_end.

- BoundaryHandler (qué pasa cuando una partícula sale del dominio):

"clip" -> recorta la posición y anula la velocidad

en esa dimensión.

"reflect" -> refleja la posición e invierte la velocidad.

"random_reset" -> reubica aleatoriamente y anula la velocidad.

```
    - Límite de velocidad v_max (fracción del rango de cada
variable).

    - StoppingCriterion: paro fuerte (máx. iteraciones), paro débil

      (estancamiento del mejor global) y colapso de diversidad
euclidiana

      (opcional, desactivado por defecto).

    - Se registra por iteración: mejor global, fitness medio del
enjambre y

      diversidad euclidiana (distancia media al centroide,
normalizada).


"""


# Importaciones Necesarias para el código explicadas en anteriores
actividadesv

from __future__ import annotations


from dataclasses import dataclass, field

from typing import Callable, Protocol, Sequence


import numpy as np

from numpy.random import Generator



"""

1. Especificación del problema (función objetivo + dominio)

_____________________________________________________________

"""


@dataclass(frozen=True)

class ProblemSpec:
```

```
    """Describe un problema de minimización con dominio tipo caja.

    Attributes:
        name: Identificador del problema.
        func: Función objetivo vectorizada: (n_pop, n_vars) -> (n_pop,).
        bounds: Límites [(min, max), ...] por variable.
        known_optimum: Valor óptimo global conocido (None si se desconoce).
        optimum_position: Posición del óptimo (solo para marcarlo en gráficas).
    """

    name: str
    func: Callable[[np.ndarray], np.ndarray]
    bounds: Sequence[tuple[float, float]]
    known_optimum: float | None = None
    optimum_position: tuple[float, ...] | None = None

    @property
    def n_vars(self) -> int:
        return len(self.bounds)


def _rosenbrock(x: np.ndarray) -> np.ndarray:
    """Rosenbrock: sum_i [(1 - x_i)^2 + 100 (x_{i+1} - x_i^2)^2].
    Para n=2: (1-x)^2 + 100 (y - x^2)^2. Óptimo global f(1,...,1) = 0 en un
    valle curvo y estrecho (difícil de recorrer: reto de explotación)."""
```

```
    return np.sum((1.0 - x[:, :-1]) ** 2 + 100.0 * (x[:, 1:] - x[:,
:-1] ** 2) ** 2, axis=1)



def make_rosenbrock(n_dims: int = 2) -> ProblemSpec:
    """Rosenbrock, x_i en [-5, 5]."""
    return ProblemSpec(
        name=f"rosenbrock_{n_dims}d",
        func=_rosenbrock,
        bounds=[(-5.0, 5.0)] * n_dims,
        known_optimum=0.0,
        optimum_position=(1.0,) * n_dims,
    )



def make_michalewicz(n_dims: int = 2, m: int = 10) -> ProblemSpec:
    """Michalewicz: f(x) = -sum_i sin(x_i) * sin^{2m}(i * x_i^2 /
pi),
    x_i en [0, pi]. Muchos mínimos locales y crestas muy angostas
para
    m grande (reto de exploración). Para n=2, m=10 el óptimo
conocido es
    f ~= -1.801303 en (2.2029, 1.5708)."""
    idx = np.arange(1, n_dims + 1, dtype=np.float64)

    def _michalewicz(x: np.ndarray) -> np.ndarray:
        return -np.sum(np.sin(x) * np.sin(idx * x**2 / np.pi) ** (2
* m), axis=1)

    is_reference_case = (n_dims == 2 and m == 10)
    return ProblemSpec(
```

```
        name=f"michalewicz_{n_dims}d",
        func=_michalewicz,
        bounds=[(0.0, float(np.pi))] * n_dims,
        known_optimum=-1.801303410098553 if is_reference_case else
None,
        optimum_position=(2.20290551, 1.57079633) if
is_reference_case else None,
    )



"""
2. Configuración del algoritmo
_____________________________________________________________
"""



@dataclass
class PSOConfig:
    """Hiperparámetros y opciones del PSO.

    Attributes:
        swarm_size: Número de partículas (N).
        max_iterations: Criterio de paro FUERTE (tope duro de
iteraciones).
        w: Peso de inercia (usado por el esquema "constant").
        c1: Coeficiente cognitivo (atracción hacia pbest).
        c2: Coeficiente social (atracción hacia lbest).
        topology: Estrategia de vecindad registrada en TOPOLOGIES.
        neighborhood_k: Tamaño de vecindad. "ring": vecinos por
CADA lado
```

```
            (k=1 -> vecindad de 3 con la propia). "euclidean_knn": número de
            vecinos más cercanos (además de la propia partícula).
            Ignorado por "global".
        inertia_schedule: Esquema registrado en INERTIA_SCHEDULES.
        w_start, w_end: Extremos de "linear_decay".
        boundary_handling: Estrategia registrada en BOUNDARY_HANDLERS.
        v_max_frac: Límite de velocidad como fracción del rango de cada
            variable; también define el rango de velocidades iniciales
            (uniforme en [-v_max, v_max]).
        stagnation_iterations: Paro DÉBIL: iteraciones sin mejora
            significativa del mejor global (0 = desactivado).
        stagnation_epsilon: Mejora mínima considerada significativa.
        diversity_epsilon: Paro OPCIONAL por colapso de diversidad euclidiana
            normalizada (0.0 = desactivado).
        success_tolerance: Tolerancia |f_encontrado - f_óptimo| para éxito.
        seed: Semilla para reproducibilidad.
    """

    swarm_size: int = 40
    max_iterations: int = 300
    w: float = 0.7
    c1: float = 1.5
    c2: float = 1.5
    topology: str = "ring"
```

```
    neighborhood_k: int = 1
    inertia_schedule: str = "constant"
    w_start: float = 0.9
    w_end: float = 0.4
    boundary_handling: str = "clip"
    v_max_frac: float = 0.2
    stagnation_iterations: int = 50
    stagnation_epsilon: float = 1e-10
    diversity_epsilon: float = 0.0
    success_tolerance: float = 1e-3
    seed: int | None = None



"""
3. Estrategias: topología de vecindad (Protocol + registry)
_____________________________________________________________
"""
# Contrato de la estreategia de vecindad
class NeighborhoodTopology(Protocol):
    def __call__(
        self, positions: np.ndarray, pbest_pos: np.ndarray,
pbest_fit: np.ndarray, config: PSOConfig
    ) -> np.ndarray:
        """Devuelve, para cada partícula, el índice de la partícula
cuyo
        pbest es el mejor de su vecindad (lbest). Forma:
(n_particles,)."""
        ...
```

```
def topology_global(
    positions: np.ndarray, pbest_pos: np.ndarray, pbest_fit:
np.ndarray, config: PSOConfig
) -> np.ndarray:
    """Vecindad = todo el enjambre (gbest). Convergencia rápida,
mayor riesgo
    de convergencia prematura."""
    return np.full(len(pbest_fit), int(np.argmin(pbest_fit)),
dtype=np.int64)



def topology_ring(
    positions: np.ndarray, pbest_pos: np.ndarray, pbest_fit:
np.ndarray, config: PSOConfig
) -> np.ndarray:
    """Anillo (lbest clásico): la vecindad de la partícula i son
las
    partículas i-k, ..., i+k (índices módulo N), incluida ella
misma. La
    información se propaga lentamente por el anillo, preservando
diversidad."""
    n = len(pbest_fit)
    k = max(1, config.neighborhood_k)
    offsets = np.arange(-k, k + 1)
    neighbor_idx = (np.arange(n)[:, None] + offsets[None, :]) % n
# (n, 2k+1)
    neighbor_fit = pbest_fit[neighbor_idx]
# (n, 2k+1)
    best_local = np.argmin(neighbor_fit, axis=1)
# (n,)
    return neighbor_idx[np.arange(n), best_local]
```

```
def topology_euclidean_knn(
    positions: np.ndarray, pbest_pos: np.ndarray, pbest_fit:
np.ndarray, config: PSOConfig
) -> np.ndarray:
    """Vecindad espacial: la partícula i y sus k vecinos más
cercanos en
    distancia euclidiana (calculada sobre las posiciones actuales).
La
    vecindad cambia dinámicamente conforme el enjambre se mueve."""
    n = len(pbest_fit)
    k = min(max(1, config.neighborhood_k), n - 1)
    diffs = positions[:, None, :] - positions[None, :, :]
# (n, n, d)
    dists = np.linalg.norm(diffs, axis=2)
# (n, n)
    # k+1 más cercanos (incluye a la propia partícula, a distancia
0)
    nearest = np.argsort(dists, axis=1)[:, : k + 1]
# (n, k+1)
    neighbor_fit = pbest_fit[nearest]
    best_local = np.argmin(neighbor_fit, axis=1)
    return nearest[np.arange(n), best_local]


# Listado de estrategias de la Topología
TOPOLOGIES: dict[str, NeighborhoodTopology] = {
    "ring": topology_ring,
    "global": topology_global,
    "euclidean_knn": topology_euclidean_knn,
}
```

```
"""
4. Estrategia: esquema de inercia (Protocol + registry)
_____________________________________________________________
"""


# Estrategia: Patrón de la Inercia
class InertiaSchedule(Protocol):
    def __call__(self, iteration: int, config: PSOConfig) -> float:
        """Devuelve el peso de inercia w a usar en la iteración
dada."""
        ...



def inertia_constant(iteration: int, config: PSOConfig) -> float:
    """w constante (config.w)."""
    return config.w



def inertia_linear_decay(iteration: int, config: PSOConfig) ->
float:
    """w decae linealmente de w_start a w_end a lo largo de
max_iterations:
    w ALTO al inicio favorece exploración; w BAJO al final favorece
    explotación (afinar la solución)."""
    frac = min(1.0, iteration / max(1, config.max_iterations))
    return config.w_start + (config.w_end - config.w_start) * frac


# Listado de Estrategias de la Inercia
INERTIA_SCHEDULES: dict[str, InertiaSchedule] = {
```

```
    "constant": inertia_constant,
    "linear_decay": inertia_linear_decay,
}


"""
5. Estrategia: manejo de fronteras (Protocol + registry)
_____________________________________________________________
"""


# Estrategia: Manejo de Desborde
class BoundaryHandler(Protocol):
    def __call__(
        self, positions: np.ndarray, velocities: np.ndarray, low:
np.ndarray, high: np.ndarray, rng: Generator
    ) -> tuple[np.ndarray, np.ndarray]:
        """Corrige posiciones (y velocidades) para respetar [low,
high]."""
        ...



def boundary_clip(positions, velocities, low, high, rng):
    """Recorta la posición a la frontera y anula la velocidad en
las
    dimensiones donde se salió (evita que la partícula "empuje" el
borde)."""
    out = (positions < low) | (positions > high)
    positions = np.clip(positions, low, high)
    velocities = np.where(out, 0.0, velocities)
    return positions, velocities
```

```
def boundary_reflect(positions, velocities, low, high, rng):
    """Refleja la posición hacia adentro e invierte la velocidad en las
    dimensiones donde se salió (rebote)."""
    below = positions < low
    above = positions > high
    positions = np.where(below, 2 * low - positions, positions)
    positions = np.where(above, 2 * high - positions, positions)
    velocities = np.where(below | above, -velocities, velocities)
    positions = np.clip(positions, low, high)  # salvaguarda por rebotes extremos
    return positions, velocities



def boundary_random_reset(positions, velocities, low, high, rng):
    """Reubica aleatoriamente (uniforme en el dominio) las coordenadas que
    se salieron y anula su velocidad (reinyecta diversidad)."""
    out = (positions < low) | (positions > high)
    if out.any():
        random_pos = low + rng.random(positions.shape) * (high - low)
        positions = np.where(out, random_pos, positions)
        velocities = np.where(out, 0.0, velocities)
    return positions, velocities

# Listado de Estrategias
BOUNDARY_HANDLERS: dict[str, BoundaryHandler] = {
    "clip": boundary_clip,
```

```
    "reflect": boundary_reflect,
    "random_reset": boundary_random_reset,
}


"""
6. Métrica de diversidad (distancia euclidiana)
_____________________________________________________________
"""

def swarm_diversity_euclidean(positions: np.ndarray, bounds_arr:
np.ndarray) -> float:
    """Distancia euclidiana MEDIA de las partículas al centroide
del
    enjambre, normalizada por la diagonal del dominio (~0 =
enjambre
    colapsado; mayor = más disperso)."""
    centroid = positions.mean(axis=0)
    mean_dist = float(np.linalg.norm(positions - centroid,
axis=1).mean())
    diagonal = float(np.linalg.norm(bounds_arr[:, 1] -
bounds_arr[:, 0]))
    return mean_dist / diagonal if diagonal > 0 else 0.0


"""
7. Criterios de paro (combinables)
_____________________________________________________________
"""
# Estregia: Criterios de Paro
class StoppingCriterion(Protocol):
```

```
    def should_stop(self, iteration: int, best_history:
list[float], positions: np.ndarray) -> bool:
        ...



@dataclass
class MaxIterationsStopping:
    """Paro FUERTE: tope duro de iteraciones."""

    max_iterations: int

    def should_stop(self, iteration: int, best_history:
list[float], positions: np.ndarray) -> bool:
        return iteration >= self.max_iterations



@dataclass
class StagnationStopping:
    """Paro DÉBIL: el mejor global no mejora más que `epsilon`
durante
    `patience` iteraciones consecutivas (0 = desactivado)."""

    patience: int
    epsilon: float

    def should_stop(self, iteration: int, best_history:
list[float], positions: np.ndarray) -> bool:
        if self.patience <= 0 or len(best_history) <=
self.patience:
            return False
        recent = best_history[-(self.patience + 1):]
```

```
        return (recent[0] - min(recent)) < self.epsilon



@dataclass
class DiversityStopping:
    """Paro OPCIONAL: diversidad euclidiana normalizada <
epsilon."""

    epsilon: float
    bounds_arr: np.ndarray

    def should_stop(self, iteration: int, best_history:
list[float], positions: np.ndarray) -> bool:
        if self.epsilon <= 0:
            return False
        return swarm_diversity_euclidean(positions,
self.bounds_arr) < self.epsilon



@dataclass
class CombinedStopping:
    """Se detiene si CUALQUIERA de los criterios se cumple."""

    criteria: list[StoppingCriterion]

    def should_stop(self, iteration: int, best_history:
list[float], positions: np.ndarray) -> bool:
        return any(c.should_stop(iteration, best_history,
positions) for c in self.criteria)
```

```
def build_stopping_criterion(config: PSOConfig, bounds_arr:
np.ndarray) -> CombinedStopping:
    return CombinedStopping(
        criteria=[

MaxIterationsStopping(max_iterations=config.max_iterations),

StagnationStopping(patience=config.stagnation_iterations,
epsilon=config.stagnation_epsilon),
            DiversityStopping(epsilon=config.diversity_epsilon,
bounds_arr=bounds_arr),
        ]
    )



"""
8. Resultado de una corrida
_____________________________________________________________
"""



@dataclass
class RunResult:
    """Resultado de una corrida completa del PSO."""

    best_x: np.ndarray
    best_fitness: float
    iterations_run: int
    convergence_best: list[float] = field(default_factory=list)    #
mejor global por iteración
```

```
    convergence_mean: list[float] = field(default_factory=list)    # fitness medio del enjambre por iteración
    diversity_history: list[float] = field(default_factory=list)  # diversidad euclidiana por iteración
    stopped_reason: str = ""


IterationCallback = Callable[[int, np.ndarray, np.ndarray], None]


"""
9. Motor PSO (agnóstico de las estrategias concretas)
_____________________________________________________________
"""


class PSO:
    """Motor del PSO (actualización síncrona). Obtiene topología, esquema de
    inercia y manejo de fronteras de los registries según `config`, por lo
    que cambiarlos en vivo es solo cambiar un campo de PSOConfig."""

    def __init__(self, problem: ProblemSpec, config: PSOConfig) -> None:
        self.problem = problem
        self.config = config
        self.low = np.array([b[0] for b in problem.bounds], dtype=np.float64)
        self.high = np.array([b[1] for b in problem.bounds], dtype=np.float64)
```

```
        self.bounds_arr = np.array(problem.bounds,
dtype=np.float64)

        self.v_max = config.v_max_frac * (self.high - self.low)


        self.topology_fn = self._lookup(TOPOLOGIES,
config.topology, "topología")

        self.inertia_fn = self._lookup(INERTIA_SCHEDULES,
config.inertia_schedule, "esquema de inercia")

        self.boundary_fn = self._lookup(BOUNDARY_HANDLERS,
config.boundary_handling, "manejo de fronteras")


    @staticmethod

    def _lookup(registry: dict, key: str, label: str):

        try:

            return registry[key]

        except KeyError as e:

            raise ValueError(f"Estrategia de {label} '{key}' no
registrada. Disponibles: {list(registry)}") from e


    def _evaluate(self, positions: np.ndarray) -> np.ndarray:

        return self.problem.func(positions)


    def run(self, rng: Generator | None = None, callback:
IterationCallback | None = None) -> RunResult:

        """Ejecuta el PSO completo."""

        cfg = self.config # Configuraciones

        rng = rng if rng is not None else
np.random.default_rng(cfg.seed) # Inicialización con o sin semilla

        stopping = build_stopping_criterion(cfg, self.bounds_arr) #
Criterio de paro
```

```
        n, d = cfg.swarm_size, self.problem.n_vars # Crear n
partículas en d dimensiones

        positions = self.low + rng.random((n, d)) * (self.high -
self.low) # Inicializar las partículas en puntos aleatorios del
plano de búsqueda

        velocities = rng.uniform(-self.v_max, self.v_max, size=(n,
d)) # Velocidades aleatorias en el rango [-v_max, v_max]

        fitness = self._evaluate(positions) # Evaluamos el fitness
de cada posición de las partículas


        pbest_pos = positions.copy() # Guardamos las posicioens

        pbest_fit = fitness.copy() # Guardamos los fitness de cada
posición


        # Diccionarios para estadística

        convergence_best: list[float] = []

        convergence_mean: list[float] = []

        diversity_history: list[float] = []


        # Contador de Iteraciones

        iteration = 0

        stopped_reason = "max_iterations" # Razón de paro por
defecto en iteraciones


        # Bucle principal

        while True:

            g_idx = int(np.argmin(pbest_fit)) # Obtenemos el mejor
(Fitness más bajo en minimización)

            convergence_best.append(float(pbest_fit[g_idx])) #
Mejor fitness

            convergence_mean.append(float(fitness.mean())) #
Guardamos la media de la conversión
```

```
diversity_history.append(swarm_diversity_euclidean(positions,
self.bounds_arr)) # Guardamos en el historial la diversidad


            # Función auxiliar para captura de snapshots y logs

            if callback is not None:

                callback(iteration, positions, fitness)


            if stopping.should_stop(iteration, convergence_best,
positions): # Evaluación de alguno de los criterios de paro

                if iteration >= cfg.max_iterations: # Por máximo de
iteraciones

                    stopped_reason = "max_iterations"

                elif cfg.diversity_epsilon > 0 and
diversity_history[-1] < cfg.diversity_epsilon: # Por pérdida de
diversidad

                    stopped_reason = "diversity_collapse"

                else:

                    stopped_reason = "stagnation" # Por
estancamienot

                break


            # --- Actualización síncrona de velocidad y posición
---

            w = self.inertia_fn(iteration, cfg) # Cálculo de la
inercia

            lbest_idx = self.topology_fn(positions, pbest_pos,
pbest_fit, cfg) # Mejor índice de vecindad

            lbest_pos = pbest_pos[lbest_idx] # Guardamos el mejor
de la vecindad


            r1 = rng.random((n, d)) #

            r2 = rng.random((n, d))
```

```
            velocities = (
                w * velocities # Inercia
                + cfg.c1 * r1 * (pbest_pos - positions) #
Componente cognitivo
                + cfg.c2 * r2 * (lbest_pos - positions) #
Componente social
            ) # Actualización de las velocidades
            velocities = np.clip(velocities, -self.v_max,
self.v_max) # Clipp de las velocidades a las máximas
            positions = positions + velocities # Actualización de
posiciones
            positions, velocities = self.boundary_fn(positions,
velocities, self.low, self.high, rng) # Manejo de límites


            fitness = self._evaluate(positions) # Evaluamos la
nueva posición
            improved = fitness < pbest_fit # Verificamos si hubo
mejoría
            pbest_pos[improved] = positions[improved] #
Actualizamos los posiciones de las mejores marcas personales
            pbest_fit[improved] = fitness[improved] # Actualizamos
los fitness de los mejores personales


            iteration += 1 # Aumentamos el contador de generaciones


        g_idx = int(np.argmin(pbest_fit)) # obtenemos el mejor de
la corrida
        return RunResult( # Retornamos los resultados
            best_x=pbest_pos[g_idx].copy(),
            best_fitness=float(pbest_fit[g_idx]),
            iterations_run=iteration,
            convergence_best=convergence_best,
```

```
            convergence_mean=convergence_mean,
            diversity_history=diversity_history,
            stopped_reason=stopped_reason,
        )


"""
10. Utilidad de éxito
_____________________________________________________________
"""


def is_success(problem: ProblemSpec, best_fitness: float,
tolerance: float) -> bool | None:
    """Éxito si |f_encontrado - f_óptimo| < tolerancia. None si no
hay óptimo conocido."""
    if problem.known_optimum is None:
        return None
    return abs(best_fitness - problem.known_optimum) < tolerance
```

main.py

```
"""
Autor: Mariscal Rodríguez Omar Jesús
Materia: Algoritmos Metaheurísticos
Profesor: Paredes López Ángel Ignasio
Actividad 5 - Optimización por Enjambre de Partículas (PSO)

Universidad de Guadalajara
Centro Universitario de Ciencias Exactas e Ingenierías

-------------------------------------------------------------------
-----

main.py

Orquesta la Actividad 5 (PSO) en dos modos:

    --mode tuning:
        Barrido de (tamaño del enjambre N, peso de inercia w,
coeficiente
        cognitivo c1, coeficiente social c2). Cada combinación se
evalúa en
        AMBAS funciones (Rosenbrock y Michalewicz) con pocas
corridas, y se
        genera un ranking de configuraciones COMUNES a ambas
(deliverable
        "Sintonización de Parámetros"). El ranking ordena por tasa
de éxito
        promedio y, como desempate, por la suma de rangos de la
media del
        mejor fitness en cada función (métrica sin escala,
comparable entre
```

```
        funciones con magnitudes muy distintas).


    --mode final (default):

        Corre la configuración elegida con >= 30 corridas
independientes

        sobre cada función. Genera, por función:

            - Tabla estadística (mejor, peor, media, desv.
estándar, éxito).

            - Curva de convergencia promedio (mejor global y
fitness medio

              del enjambre vs. iteración).

            - Scatter de las partículas en 3 momentos (inicial,
intermedia,

              final) sobre el contorno de la función, con una
corrida

              DEDICADA (semilla propia reproducible, separada de
las 30).

            - Gráfica de diversidad euclidiana vs. iteración
(extra).


Para modificar en vivo (evaluación presencial), basta cambiar un
flag:

    --topology global | ring | euclidean_knn
(--neighborhood-k)

    --inertia-schedule constant | linear_decay    (--w, --w-start,
--w-end)

    --boundary-handling clip | reflect | random_reset

    Todos solo modifican PSOConfig; pso.PSO no requiere otros
cambios.


Uso:

    python main.py --mode tuning
```

```
    python main.py --mode final --swarm-size 40 --w 0.7 --c1 1.5
--c2 1.5

    python main.py --mode final --runs 5   # corrida rápida de
prueba

"""


# Importaciones necesarias

from __future__ import annotations


import argparse

import csv

import dataclasses

import itertools

import os

import time

from dataclasses import dataclass


import matplotlib

matplotlib.use("Agg")

import matplotlib.pyplot as plt

import numpy as np


from pso import (

    PSO,

    PSOConfig,

    ProblemSpec,

    RunResult,

    is_success,

    make_michalewicz,

    make_rosenbrock,
```

```
)


# Ajustes por defecto
OUTPUT_DIR_DEFAULT = "resultados"
N_RUNS_FINAL_DEFAULT = 30



"""
Utilidades comunes
_____________________________________________________________
"""


# Contruir los problemas pedidos
def build_problems() -> dict[str, ProblemSpec]:
    return {"rosenbrock": make_rosenbrock(2), "michalewicz":
make_michalewicz(2, m=10)}


# Correr una vez el elgoritmo
def run_single(problem: ProblemSpec, config: PSOConfig, seed: int,
callback=None) -> RunResult:
    return PSO(problem,
config).run(rng=np.random.default_rng(seed), callback=callback)


# Correr varias veces el algoritmo
def run_many(problem: ProblemSpec, config: PSOConfig, n_runs: int,
base_seed: int) -> list[RunResult]:
    seeder = np.random.default_rng(base_seed)
    results = []
    for _ in range(n_runs):
        run_seed = int(seeder.integers(0, 2**31 - 1))
```

```
        results.append(run_single(problem,
dataclasses.replace(config, seed=run_seed), run_seed))
    return results


# Promedio de la curba
def mean_curve(runs: list[RunResult], attr: str) -> np.ndarray:
    """Promedia una curva por iteración entre corridas; las
corridas más
    cortas (paro débil) se extienden con su último valor."""
    curves = [getattr(r, attr) for r in runs]
    max_len = max(len(c) for c in curves)
    return np.array([c + [c[-1]] * (max_len - len(c)) for c in
curves]).mean(axis=0)


# De consola a real
def parse_float_list(text: str) -> list[float]:
    return [float(v) for v in text.split(",")]


#De consola a entero
def parse_int_list(text: str) -> list[int]:
    return [int(v) for v in text.split(",")]


# Construcción desde consola del PSO Config
def config_from_args(args: argparse.Namespace, **overrides) ->
PSOConfig:
    """Construye un PSOConfig desde los flags de CLI, con overrides
opcionales."""
    base = dict(
        swarm_size=args.swarm_size,
        max_iterations=args.max_iterations,
        w=args.w,
```

```
        c1=args.c1,
        c2=args.c2,
        topology=args.topology,
        neighborhood_k=args.neighborhood_k,
        inertia_schedule=args.inertia_schedule,
        w_start=args.w_start,
        w_end=args.w_end,
        boundary_handling=args.boundary_handling,
        v_max_frac=args.v_max_frac,
        stagnation_iterations=args.stagnation_iterations,
        stagnation_epsilon=args.stagnation_epsilon,
        diversity_epsilon=args.diversity_epsilon,
        success_tolerance=args.success_tolerance,
    )
    base.update(overrides)
    return PSOConfig(**base)


# Función para impresiones de hiperparámetros
def config_label(cfg: PSOConfig) -> str:
    w_txt = f"w={cfg.w}" if cfg.inertia_schedule == "constant" else
f"w:{cfg.w_start}->{cfg.w_end}"
    return (f"N={cfg.swarm_size}, {w_txt}, c1={cfg.c1},
c2={cfg.c2}, topología={cfg.topology}"
            f"(k={cfg.neighborhood_k}),
fronteras={cfg.boundary_handling}")



"""
Modo TUNING
_____________________________________________________________
```

```
"""

#Contenedor de datos del modo tuning

@dataclass

class TuningRow:

    swarm_size: int

    w: float

    c1: float

    c2: float

    problem: str

    mean_best: float

    std_best: float

    success_rate: float | None


# Corrida del modo tuning

def run_tuning(args: argparse.Namespace) -> list[TuningRow]:

    problems = build_problems() # Construcción de los problemas

    sizes = parse_int_list(args.tuning_swarm_sizes) # Pasamos a
enterlos los tamaños

    ws = parse_float_list(args.tuning_w) # Peso de inercia

    c1s = parse_float_list(args.tuning_c1) # Peso del factor
cognitivo

    c2s = parse_float_list(args.tuning_c2) # Peso del factor social


    # Si el usuario elige linear_decay, el valor de w que se barre
no se usa (se ignoran los valores del barrido y se usan
w_start/w_end en su lugar).

    if args.inertia_schedule != "constant":

        print("AVISO: el tuning de 'w' asume inercia constante; con
linear_decay el valor de w del "

              "barrido no se usa (se usan --w-start/--w-end).\n")
```

```
    combos = list(itertools.product(sizes, ws, c1s, c2s)) # Lista
de combinaciones

    total_runs = len(combos) * len(problems) * args.tuning_runs #
Cálculo del total de runs necerias

    print(f"Modo TUNING: {len(combos)} combinaciones x
{len(problems)} funciones x "

          f"{args.tuning_runs} corridas = {total_runs}
ejecuciones\n")


    rows: list[TuningRow] = []

    t_start = time.time() # Tiempo para logs

    for i, (n, w, c1, c2) in enumerate(combos, start=1): # Bucle
principal

        cfg = config_from_args(args, swarm_size=n, w=w, c1=c1,
c2=c2) # Configuraciones esenciales

        for pname, problem in problems.items(): # Por combinación

            runs = run_many(problem, cfg, args.tuning_runs,
base_seed=args.seed) # Correr el algoritmo

            vals = np.array([r.best_fitness for r in runs]) #
Obtner los mejores fitness

            succ = [is_success(problem, v, args.success_tolerance)
for v in vals] # Calcular el porcentaje de éxito basado en el
mínimo conocido

            rows.append(TuningRow( # Estadística final

                swarm_size=n, w=w, c1=c1, c2=c2, problem=pname,

                mean_best=float(vals.mean()),

                std_best=float(vals.std(ddof=1)) if len(vals) > 1
else 0.0,

                success_rate=float(np.mean(succ)) if all(s is not
None for s in succ) else None,

            ))

        if i % 10 == 0 or i == len(combos):
```

```
            print(f"  [{i}/{len(combos)}] combinaciones evaluadas
({time.time() - t_start:.0f}s)") # Impresiones finales

    return rows


#Contenedor de datos del modo tuning

@dataclass

class CombinedRow:

    swarm_size: int

    w: float

    c1: float

    c2: float

    mean_best: dict[str, float]

    success: dict[str, float]

    combined_success: float

    rank_sum: float



def combine_tuning_rows(rows: list[TuningRow]) ->
list[CombinedRow]:

    """Agrupa por configuración y calcula el ranking común a ambas
funciones."""

    groups: dict[tuple, dict[str, TuningRow]] = {}

    for r in rows:

        groups.setdefault((r.swarm_size, r.w, r.c1, r.c2),
{})[r.problem] = r


    problems = sorted({r.problem for r in rows})

    # Rango (1 = mejor) de cada configuración por función, según la
media del mejor fitness

    ranks: dict[str, dict[tuple, int]] = {}

    for p in problems:
```

```
        ordered = sorted(groups, key=lambda k:
groups[k][p].mean_best)
        ranks[p] = {k: pos + 1 for pos, k in enumerate(ordered)}


    combined = [] # Combinaciones
    for key, per_problem in groups.items(): # Por combinación
        success = {p: (per_problem[p].success_rate or 0.0) for p in
problems}
        combined.append(CombinedRow(
            swarm_size=key[0], w=key[1], c1=key[2], c2=key[3],
            mean_best={p: per_problem[p].mean_best for p in
problems},
            success=success,

combined_success=float(np.mean(list(success.values()))),
            rank_sum=float(sum(ranks[p][key] for p in problems)),
        ))
    combined.sort(key=lambda c: (-c.combined_success, c.rank_sum))
    return combined


#Función auxiliar de impresión del modo tuning
def print_tuning_ranking(combined: list[CombinedRow], top: int =
15) -> None:
    problems = list(combined[0].mean_best)
    header = f"{'Pos':>3} {'N':>4} {'w':>5} {'c1':>5} {'c2':>5}
{'Éxito comb.':>12} "
    header += " ".join(f"{'Éxito ' + p[:5]:>13} {'Media ' +
p[:5]:>13}" for p in problems)
    print(f"\nTop {top} configuraciones comunes (orden: éxito
combinado, luego suma de rangos)")
    print(header)
    print("-" * len(header))
```

```
    for pos, c in enumerate(combined[:top], start=1):
        line = f"{pos:>3} {c.swarm_size:>4} {c.w:>5.2f}
{c.c1:>5.2f} {c.c2:>5.2f} {c.combined_success:>11.0%} "
        line += " ".join(f"{c.success[p]:>13.0%}
{c.mean_best[p]:>13.6f}" for p in problems)
        print(line)


#Función auxiliar para la exportación de resultados del modo tuning
para el reporte
def export_tuning_csv(combined: list[CombinedRow], path: str) ->
None:
    problems = list(combined[0].mean_best)
    with open(path, "w", newline="", encoding="utf-8") as f:
        writer = csv.writer(f)
        header = ["posicion", "N", "w", "c1", "c2",
"exito_combinado", "suma_rangos"]
        for p in problems:
            header += [f"exito_{p}", f"media_mejor_{p}"]
        writer.writerow(header)
        for pos, c in enumerate(combined, start=1):
            row = [pos, c.swarm_size, c.w, c.c1, c.c2,
f"{c.combined_success:.4f}", f"{c.rank_sum:.0f}"]
            for p in problems:
                row += [f"{c.success[p]:.4f}",
f"{c.mean_best[p]:.8f}"]
            writer.writerow(row)
    print(f"\nTabla completa de sintonización exportada a: {path}")
    print("Elige una configuración con buen éxito combinado y corre
--mode final con esos valores.")


"""
Modo FINAL
```

```
"""


# Contenedor de datos para el modo final
@dataclass
class StatsRow:
    problem: str
    best: float
    worst: float
    mean: float
    std: float
    success_rate: float | None
    mean_iterations: float


# Computar estadísticas del modo fincla
def compute_stats(problem: ProblemSpec, runs: list[RunResult],
tolerance: float) -> StatsRow:
    vals = np.array([r.best_fitness for r in runs])
    succ = [is_success(problem, v, tolerance) for v in vals]
    return StatsRow(
        problem=problem.name,
        best=float(vals.min()),
        worst=float(vals.max()),
        mean=float(vals.mean()),
        std=float(vals.std(ddof=1)) if len(vals) > 1 else 0.0,
        success_rate=float(np.mean(succ)) if all(s is not None for
s in succ) else None,
        mean_iterations=float(np.mean([r.iterations_run for r in
runs])),
    )
```

```
# Impresión del modo final

def print_stats_table(rows: list[StatsRow]) -> None:

    header = (f"{'Problema':<16} {'Mejor':>14} {'Peor':>14}
{'Media':>14} {'Desv.Std':>14} "

              f"{'% Éxito':>8} {'Iter. media':>12}")

    print("\n" + header)

    print("-" * len(header))

    for r in rows:

        s = f"{r.success_rate:.0%}" if r.success_rate is not None
else "N/A"

        print(f"{r.problem:<16} {r.best:>14.8f} {r.worst:>14.8f}
{r.mean:>14.8f} {r.std:>14.8f} "

              f"{s:>8} {r.mean_iterations:>12.1f}")


#Exportar a csv la ejecución final

def export_stats_csv(rows: list[StatsRow], path: str) -> None:

    with open(path, "w", newline="", encoding="utf-8") as f:

        writer = csv.writer(f)

        writer.writerow(["problema", "mejor", "peor", "media",
"desv_std", "tasa_exito", "iter_media"])

        for r in rows:

            writer.writerow([

                r.problem, f"{r.best:.10f}", f"{r.worst:.10f}",
f"{r.mean:.10f}", f"{r.std:.10f}",

                f"{r.success_rate:.4f}" if r.success_rate is not
None else "N/A", f"{r.mean_iterations:.2f}",

            ])

    print(f"\nTabla estadística exportada a: {path}")


# Gráfica de la convergencia
```

```
def plot_convergence(problem: ProblemSpec, runs: list[RunResult],
path: str, label: str) -> None:

    """Mejor global promedio y fitness medio del enjambre promedio
vs. iteración."""

    best_curve = mean_curve(runs, "convergence_best")

    mean_curve_swarm = mean_curve(runs, "convergence_mean")



    fig, ax = plt.subplots(figsize=(10, 6))

    ax.plot(best_curve, linewidth=1.8, label="Mejor global
(promedio de corridas)")

    ax.plot(mean_curve_swarm, linewidth=1.5, linestyle="--",
label="Fitness medio del enjambre (promedio de corridas)")

    if problem.known_optimum is not None:

        ax.axhline(problem.known_optimum, color="black",
linestyle=":", linewidth=1, label="Óptimo global")

    if min(best_curve.min(), mean_curve_swarm.min()) > 0:

        ax.set_yscale("log")

    ax.set_xlabel("Iteración")

    ax.set_ylabel("Fitness")

    ax.set_title(f"Convergencia promedio —
{problem.name}\n({label})")

    ax.legend(fontsize=8)

    ax.grid(True, alpha=0.3, which="both")

    fig.tight_layout()

    fig.savefig(path, dpi=150)

    plt.close(fig)

    print(f"Gráfica de convergencia guardada: {path}")


# Graficar diversidad

def plot_diversity(problem: ProblemSpec, runs: list[RunResult],
path: str, label: str) -> None:
```

```
    fig, ax = plt.subplots(figsize=(10, 6))

    ax.plot(mean_curve(runs, "diversity_history"), linewidth=1.8,
color="tab:green")

    ax.set_xlabel("Iteración")

    ax.set_ylabel("Diversidad euclidiana normalizada (promedio)")

    ax.set_title(f"Diversidad del enjambre vs. iteración —
{problem.name}\n({label})")

    ax.grid(True, alpha=0.3)

    fig.tight_layout()

    fig.savefig(path, dpi=150)

    plt.close(fig)

    print(f"Gráfica de diversidad guardada: {path}")


# Captura de resumen en momentos específicos de la corrida

def capture_snapshots(problem: ProblemSpec, config: PSOConfig,
seed: int) -> list[tuple[int, np.ndarray]]:

    """Corrida dedicada (independiente de las N estadísticas) que
captura las

    posiciones del enjambre en cada iteración y devuelve 3
snapshots:

    inicial, intermedia (mitad de las iteraciones realmente
ejecutadas) y final."""

    history: list[tuple[int, np.ndarray]] = []

    run_single(problem, config, seed, callback=lambda it, pos, fit:
history.append((it, pos.copy())))

    return [history[0], history[len(history) // 2], history[-1]]


# Graficar la distribución espacial

def plot_spatial_distribution(

    problem: ProblemSpec, snapshots: list[tuple[int, np.ndarray]],
path: str, label: str

) -> None:
```

```
    """Scatter de las partículas en 3 momentos sobre el contorno de
la función."""

    (x_lo, x_hi), (y_lo, y_hi) = problem.bounds[0],
problem.bounds[1]

    xs, ys = np.linspace(x_lo, x_hi, 300), np.linspace(y_lo, y_hi,
300)

    xx, yy = np.meshgrid(xs, ys)

    zz = problem.func(np.column_stack([xx.ravel(),
yy.ravel()])).reshape(xx.shape)


    # Rosenbrock abarca varios órdenes de magnitud: contorno en
escala log para ver el valle

    use_log = zz.min() >= 0

    z_plot = np.log10(zz + 1.0) if use_log else zz

    cbar_label = "log10(f + 1)" if use_log else "f(x, y)"


    titles = ["Generación inicial", "Generación intermedia",
"Generación final"]

    fig, axes = plt.subplots(1, 3, figsize=(19, 6), sharex=True,
sharey=True)

    contour = None

    for ax, (it, pos), title in zip(axes, snapshots, titles):

        contour = ax.contourf(xx, yy, z_plot, levels=40,
cmap="viridis", alpha=0.85)

        # El óptimo se dibuja DEBAJO de las partículas (zorder
menor): al final de la

        # corrida las partículas se concentran sobre él y, si no,
quedarían tapadas.

        if problem.optimum_position is not None:

            ax.scatter(*problem.optimum_position[:2], marker="*",
c="gold", edgecolors="black",

                       s=420, zorder=4, label="Óptimo global")
```

```
        ax.scatter(pos[:, 0], pos[:, 1], c="red",
edgecolors="white", s=22, linewidths=0.5, zorder=6,
                   label="Partículas")
        ax.set_title(f"{title} (iteración {it})")
        ax.set_xlabel("x")
        ax.set_xlim(x_lo, x_hi)
        ax.set_ylim(y_lo, y_hi)
    axes[0].set_ylabel("y")
    axes[0].legend(loc="upper left", fontsize=8)
    fig.suptitle(f"Distribución espacial del enjambre —
{problem.name}\n({label})")
    fig.colorbar(contour, ax=axes, shrink=0.85, label=cbar_label)
    fig.savefig(path, dpi=150)
    plt.close(fig)
    print(f"Gráfica de distribución espacial guardada: {path}")


# Correr el modo final
def run_final(args: argparse.Namespace) -> None:
    config = config_from_args(args) # Cargar configuraciones
    label = config_label(config) # Callable del headder de
hiper-parámetros
    problems = build_problems() # Construcción de problemas


    # Impresión Inicial
    print("=" * 78)
    print("Actividad 5 - Optimización por Enjambre de Partículas
(modo FINAL)")
    print(f"Corridas por función: {args.runs}")
    print(label)
    print("=" * 78)
```

```
    os.makedirs(args.output_dir, exist_ok=True) # Carpeta de
exportación

    stats_rows: list[StatsRow] = [] # Estadísticas


    for key, problem in problems.items(): # Blucle principal

        print(f"\nEjecutando {problem.name} ({args.runs}
corridas)...", end="", flush=True) # Inicio del modo final

        t0 = time.time() # Medición del tiempo

        runs = run_many(problem, config, args.runs,
base_seed=args.seed) # Correr varias veces el algoritmo

        print(f" listo ({time.time() - t0:.1f}s)") # Final del
tiempo

        stats_rows.append(compute_stats(problem, runs,
args.success_tolerance)) # Cálculo de estadísticas


        plot_convergence(problem, runs,
os.path.join(args.output_dir, f"convergencia_{key}.png"), label) #
Graficar Convergencia

        plot_diversity(problem, runs, os.path.join(args.output_dir,
f"diversidad_{key}.png"), label) # Graficar Diversidad


        viz_seed = args.seed + 999_999 # Semilla aleatoria lejos
para snapshot

        print(f"Corrida dedicada para distribución espacial de
{problem.name} (semilla={viz_seed})")

        snapshots = capture_snapshots(problem, config, viz_seed) #
Captura de snapshots

        plot_spatial_distribution( # Graficar distribución espacial
de la snapshot

            problem, snapshots, os.path.join(args.output_dir,
f"distribucion_espacial_{key}.png"), label

        )
```

```
    print_stats_table(stats_rows) # Impresión de estadísticas

    export_stats_csv(stats_rows, os.path.join(args.output_dir,
"tabla_estadistica_pso.csv")) # Exportar estadísticas

    print("\nExperimento final completo. Resultados en:",
os.path.abspath(args.output_dir))



"""
CLI / punto de entrada
_____________________________________________________________
"""


# Pasar desde consola
def parse_args() -> argparse.Namespace:

    p = argparse.ArgumentParser(description="Actividad 5 - PSO:
tuning y experimento final")

    p.add_argument("--mode", type=str, default="final",
choices=["tuning", "final"])


    # Comunes

    p.add_argument("--seed", type=int, default=42)

    p.add_argument("--max-iterations", type=int, default=300)

    p.add_argument("--topology", type=str, default="ring",
choices=["ring", "global", "euclidean_knn"])

    p.add_argument("--neighborhood-k", type=int, default=1)

    p.add_argument("--inertia-schedule", type=str,
default="constant", choices=["constant", "linear_decay"])

    p.add_argument("--w-start", type=float, default=0.9)

    p.add_argument("--w-end", type=float, default=0.4)
```

```
    p.add_argument("--boundary-handling", type=str, default="clip",
choices=["clip", "reflect", "random_reset"])

    p.add_argument("--v-max-frac", type=float, default=0.2)

    p.add_argument("--stagnation-iterations", type=int, default=50)

    p.add_argument("--stagnation-epsilon", type=float,
default=1e-10)

    p.add_argument("--diversity-epsilon", type=float, default=0.0)

    p.add_argument("--success-tolerance", type=float, default=1e-3)

    p.add_argument("--output-dir", type=str,
default=OUTPUT_DIR_DEFAULT)


    # Parámetros del enjambre (modo final; en tuning son la base)

    p.add_argument("--runs", type=int,
default=N_RUNS_FINAL_DEFAULT)

    p.add_argument("--swarm-size", type=int, default=40)

    p.add_argument("--w", type=float, default=0.7)

    p.add_argument("--c1", type=float, default=1.5)

    p.add_argument("--c2", type=float, default=1.5)


    # Modo tuning

    p.add_argument("--tuning-runs", type=int, default=10)

    p.add_argument("--tuning-swarm-sizes", type=str,
default="20,40,80")

    p.add_argument("--tuning-w", type=str, default="0.4,0.7,0.9")

    p.add_argument("--tuning-c1", type=str, default="1.0,1.5,2.0")

    p.add_argument("--tuning-c2", type=str, default="1.0,1.5,2.0")

    return p.parse_args()


# Función main

def main() -> None:

    args = parse_args()
```

```
    if args.mode == "tuning":
        rows = run_tuning(args)
        combined = combine_tuning_rows(rows)
        print_tuning_ranking(combined)
        os.makedirs(args.output_dir, exist_ok=True)
        export_tuning_csv(combined, os.path.join(args.output_dir,
"tabla_sintonizacion_pso.csv"))
    else:
        run_final(args)



if __name__ == "__main__":
    main()
```